

DL for ENG

LECTURE 7.1

Physics-Informed Neural Networks

A network as the solution of a differential equation. No labels in the loss: the equation itself, evaluated on the network by autograd, and the optimisers you already have pointed at it.

Remus Teodorescu

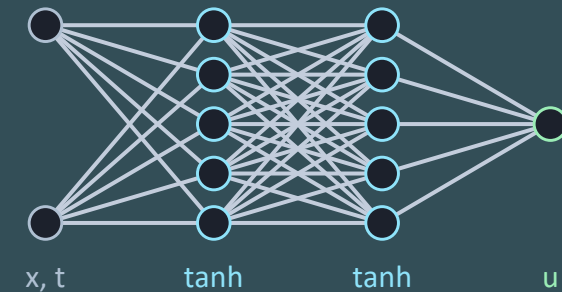
With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

ansatz · autograd · residual · collocation · Adam then L-BFGS

ONE LOOP, THE WHOLE OF PART 2



AUTOGRAD

derivatives
of u

RESIDUAL

the equation,
evaluated

LOSS

mean of the
square

*a gradient step on the
weights*

*The same loop solves heat, a battery, a flow and a grid.
What changes is one line: the residual.*

Learning Objectives

SECTIONS, IN ORDER

- 1 **PINN** — what it is, where it sits between a solver and a data fit, and where it is used
- 2 **Approximation** — the ansatz, universal approximation, smooth activations, the sampling condition
- 3 **Differentiation** — autograd on the graph, exact, and differentiable twice
- 4 **Formulation** — operator form, the four condition types, forward and inverse
- 5 **Loss construction** — the residual, soft and hard boundary terms, Neumann, initial, weights
- 6 **Residual evaluation** — collocation, the stencil, the energy method
- 7 **Training** — the loop, Adam then L-BFGS, adaptive sampling
- 8 **Challenges** — gradient imbalance, causality, sharp fronts, and the variants that answer them

ASSUMED, AND WHERE IT IS TAUGHT

autograd	L6.1, physics in the loss; Ex_02
optimisers	L6.1, Adam and L-BFGS; Ex_06
dense networks	L4.1, the perceptron to the MLP
the five PDEs	the next lecture, L7.2

The book for Part 2 is Liu, PINN with Python (2025). Chapter and equation numbers on the slides are its.

Recap

THE TEN QUESTIONS OF L6.2 — POST-TRAINING AND REINFORCEMENT LEARNING

- 1** What is transfer learning, why do the first layers transfer, and when should you unfreeze more than the head? *UDL ch. 9.3; Ex_06 notebook 03*
- 2** What task does self-supervised pre-training invent, and what does it buy an engineer with unlabelled data? *UDL ch. 9.3*
- 3** What are the three stages of post-training, and why is the third one reinforcement learning? *UDL ch. 19; [ext]*
- 4** What kind of problem is reinforcement learning, what is the policy, and how do its objects map onto control? *UDL ch. 19.1; Sutton & Barto (2018)*
- 5** What is the return, why discount, and how would you set γ for a system with a known timescale? *UDL ch. 19.2*
- 6** What is credit assignment, why does a late aggregate reward make learning hard, and what is reward hacking? *UDL ch. 19.3*
- 7** How does the policy gradient obtain a gradient from a world you cannot differentiate? *UDL ch. 19.4 – 19.5*
- 8** Imitation or reinforcement for the camera car: what decides it, and what does imitation give up? *Ex_11.1; Ross et al. (2011)*
- 9** Why does int8 quantisation barely hurt a classifier and badly hurt a regression, and which does the car run? *Ex_06 notebook 04; [ext]*
- 10** A model runs at four frames per second on a car at 2 m/s. How far does it travel blind, and what are the three ways to shorten that? *Ex_11.1; [ext]*

Physics-Informed Network

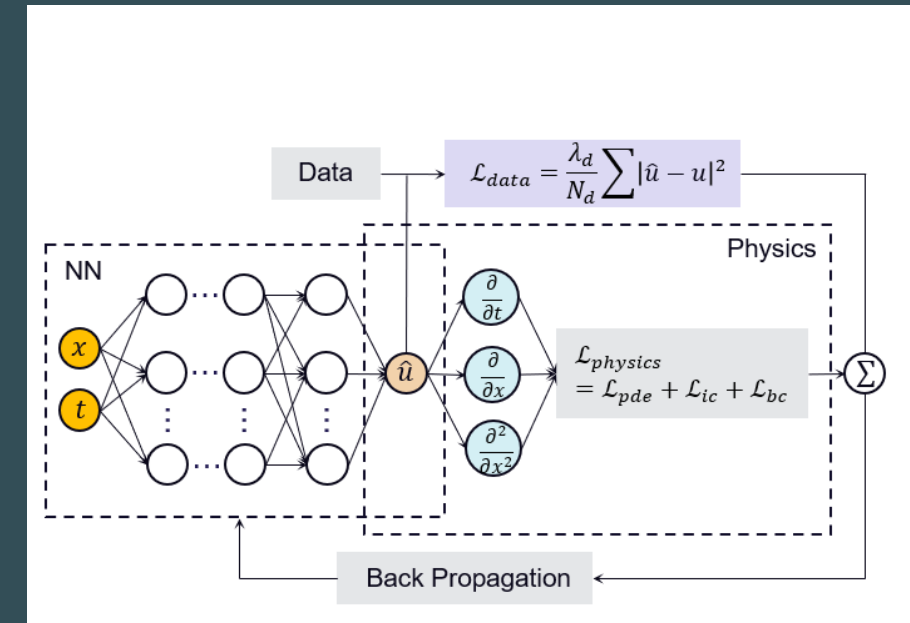
KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a universal approximator with the physics as its regulariser
- **function** — the solution field itself: $u(x, t) = N(x, t; \hat{w})$
- **architecture** — dense tanh layers, autograd, a two-part loss
- **applications** — flow, heat transfer, elasticity, inverse problems

AGAINST A DENSE NETWORK AND A MESH-BASED SOLVER

	dense network	PINN	FEM, FD
the unknown	weights	weights	nodal values
the loss	error on labels	the PDE residual	none, a solve
the data	required	optional	none
the mesh	none	none	required

THE ARCHITECTURE, AFTER RAISSI ET AL.



Two losses, one network: data on measurements, physics from the derivatives of $\hat{u}$.

Modelling Spectrum

FROM A SOLVER TO A DATA FIT, AND WHERE A PINN SITS

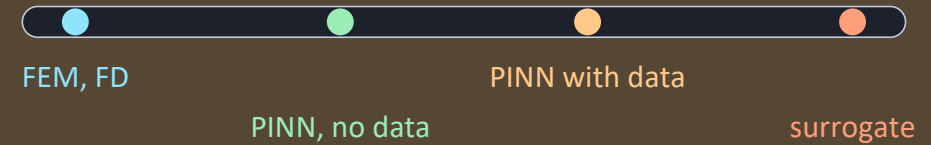
- a **mesh-based solver** — the equation, a grid, no data
- a **PINN with no data** — the equation, sampled points, no grid
- a **PINN with data** — the equation plus sparse measurements
- a **surrogate** — data only; the physics must be in the data
- the position on the bar is a choice, made by what the loss contains

TALK · 1 MIN Where on the bar does your own project sit today, and what one addition to its loss would move it one step to the left?

MESH-BASED AGAINST MESH-FREE

pure physics

pure data



	mesh-based	mesh-free
unknowns	nodal values	weights
derivatives	stencil, element	autograd
geometry	meshed	sampled
data	not used	one more term
one solution	a linear solve	a training run

A training run costs more than a linear solve; the physics regularises the low-data regime.

Applications

WHERE THE METHOD IS USED

- **fluids** — Navier–Stokes, incompressible and high-speed flow
- **solids** — linear and nonlinear elasticity, plate bending, contact
- **transport** — multiphase flow in porous media, kinetic equations
- **not only solving** — system identification and data assimilation

GOOD FIT AGAINST POOR FIT

	a good fit	a poor fit
the data	sparse, noisy, incomplete	none, and a solver exists
the unknown	a parameter as well as a field	one field, once
the geometry	awkward to mesh	a regular grid
the answer	wanted at any resolution	wanted on one grid

WHERE PART 2 TAKES IT

block	what it solves	equation
L8	steady and transient heat	Poisson, heat
L9	laminar and turbulent flow	Burgers, Navier–Stokes
L10	ionic diffusion and cells	coupled transport
L11	vision-based navigation	dynamics, on hardware
L12	power grid stability	Poisson on a graph

None of those blocks introduces a new method: each changes the operator and the conditions. And the solution is a function, so it is evaluated at any resolution afterwards without retraining.

Solution Ansatz

THE NETWORK IS THE SOLUTION, NOT A MODEL OF DATA

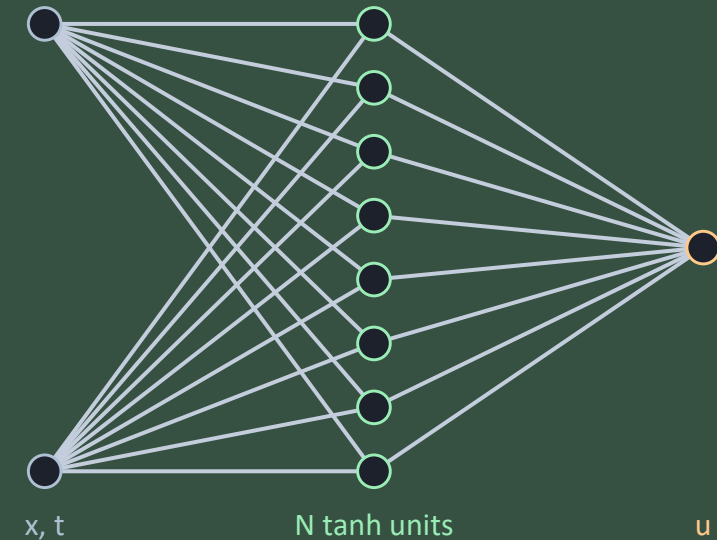
$$u(\mathbf{x}, t) = \mathcal{N}_{\tilde{w}}(\mathbf{x}, t)$$

- **input** — the coordinates $\mathbf{x}$ and t ; output — the field value there
- one network gives u at any point, with no interpolation
- **the theorem** — one hidden layer, wide enough, reaches any ε
- it promises existence; it says nothing about finding the weights

$$\sup_{\mathbf{x} \in K} \left| u(\mathbf{x}) - \sum_{j=1}^N a_j \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j) \right| < \epsilon$$

universal approximation — a continuous u on a compact K , N tanh units, any ϵ

ONE HIDDEN LAYER, N UNITS



Existence is guaranteed for this shape. In practice two or three narrower layers train better than one wide one, and the theorem does not mind.

Smooth Activations

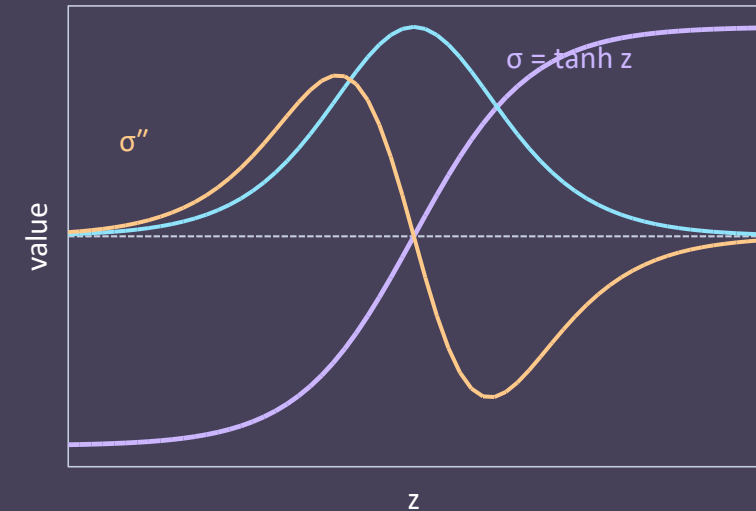
A RESIDUAL OF ORDER K NEEDS K DERIVATIVES

- the activation is the only nonlinearity — it sets the ceiling
- a second-order equation needs σ'' to exist and to be non-zero
- tanh** — bounded, monotone, differentiable to any order
- ReLU** — piecewise linear, $\sigma'' = 0$: the residual collapses

THE SECOND DERIVATIVE, BY ACTIVATION

activation	$\sigma''(z)$	second-order PDE
tanh	$-2 \tanh z (1 - \tanh^2 z)$	the default
sin, SIREN	$-\omega_0^2 \sin(\omega_0 z)$	oscillatory solutions
sigmoid	$\sigma (1 - \sigma)(1 - 2\sigma)$	saturates; avoid
ReLU	0 almost everywhere	no

TANH AND ITS FIRST TWO DERIVATIVES



All three are smooth. With ReLU the amber curve is zero wherever it is defined: a Laplacian of nothing.

Sampling Condition

NEURONS, NOT WEIGHTS, SET THE POINT COUNT

$$P^* = (p + 1) + (N_n + 1) N_L$$

$$N_{\text{sample}} \geq P^*$$

P^* — the pseudo-dimension: p inputs, N_n neurons per layer, N_L layers. The interior sample needs at least P^* points, and more is better.

- **P^* counts neurons** — the size of the space the network spans
- fewer than P^* independent points and the fit is under-determined
- **boundary and initial points are extra** — the rule is for the interior
- **P^* is far below the weight count** — by a factor of 26 to 150 here
- it is the residual, not data, that constrains the fit at each point

WORKED EXAMPLES

network	inputs	P^*	weights	problem
50×2	2	105	2 751	Poisson
100×2	3	206	10 601	heat, wave
200×4	3	808	121 601	plate, hole

Ex_07 40×2 , two inputs: $P^* = 85$

its sample 320 interior points, four times P^*

Weights and P^ are different quantities: 121 601 weights are constrained by 808 well-placed points, because a point constrains a residual, not a label.*

Automatic Differentiation

THE CHAIN RULE THROUGH THE FORWARD GRAPH

- **exact to machine precision** — no step size, no truncation
- **finite differences** — a step h ; truncation against round-off
- **symbolic** — exact, but the expressions swell with each order
- the pass that gives the optimiser its gradient also gives $\partial u / \partial x$

THE RESIDUAL'S DERIVATIVES, IN PYTORCH

```
u = net(xt)                # xt.requires_grad_(True)
g = torch.autograd.grad(u, xt, torch.ones_like(u),
                        create_graph=True)[0]
u_x, u_t = g[:, :1], g[:, 1:]
u_xx = torch.autograd.grad(u_x, xt, torch.ones_like(u_x),
                           create_graph=True)[0][:, :1]
```

THE GRAPH, TWICE BACKWARD

(x, t) inputs, with `requires_grad`

$N(x, t; \hat{w})$ the forward pass

u the output, one per point

$\partial u / \partial x, \partial u / \partial t$ first backward pass

$\partial^2 u / \partial x^2$ second backward pass

$F(u)$ the residual, assembled

*Derivatives flow back from the output.
`create_graph=True` keeps the first backward pass itself
differentiable, which is what the second one needs.*

Operator Form

ONE EQUATION FOR EVERY PROBLEM IN PART 2

$$\mathcal{F}(u; \mathbf{x}, t, \mathbf{c}) = 0, \quad \mathbf{x} \in \Omega, \quad t \in [0, t_{\text{end}}]$$

$\mathcal{F}$ — a differential operator on u ; $\mathbf{c}$ — the physical parameters; Ω — the domain

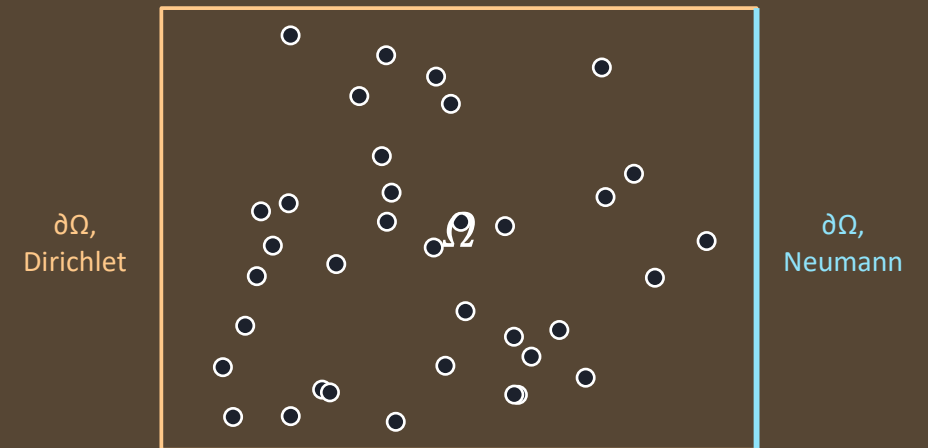
- **F is linear or nonlinear** — the network does not care which
- for a static problem t drops out; for an inverse one $\mathbf{c}$ is unknown
- the residual is $\mathcal{F}$ with the network substituted for u

Poisson $\mathcal{F} = u_{xx} + u_{yy} + f$

heat $\mathcal{F} = u_t - \alpha (u_{xx} + u_{yy})$

wave $\mathcal{F} = u_{tt} - c^2 (u_{xx} + u_{yy})$

THE DOMAIN AND ITS BOUNDARY



$\mathcal{F}$ is checked at the white points, the conditions on the edges: a value on the amber walls, a derivative on the cyan one. The points carry no boundary information at all.

Boundary and Initial Conditions

- DBC

$u(\mathbf{x}, t) = \bar{u}(\mathbf{x}, t) \quad \forall \mathbf{x} \in \partial\Omega_{DBC}$
Dirichlet — a value prescribed on part of the boundary
- NBC

$\mathcal{B}(\mathbf{x}, t, \mathbf{c}) = 0 \quad \forall \mathbf{x} \in \partial\Omega_{NBC}$
Neumann — a derivative prescribed: flux, traction, insulation
- IC

$\mathcal{I}(\mathbf{x}, 0, \mathbf{c}) = 0 \quad \forall \mathbf{x} \in \Omega$
initial condition — the field at $t = 0$
- IV

$\mathcal{V}(\mathbf{x}, 0, \mathbf{c}) = 0 \quad \forall \mathbf{x} \in \Omega$
initial velocity — its rate at $t = 0$; second order in time only

WHICH PROBLEM CARRIES WHICH

	Poisson	heat	wave
DBC	yes	yes	yes
NBC	on a flux wall	if insulated	if free
IC	—	one	one
IV	—	—	one

- B

$\partial u / \partial n$, minus its value
- I

$u(\mathbf{x}, 0)$, minus the initial field
- V

$\dot{u}(\mathbf{x}, 0)$, minus the initial rate

Four operators, one network, one autograd. The machinery does not change with the condition.

Forward and Inverse Problems

$$u_t + \mathcal{N}[u; \lambda] = 0$$

Raissi's general form: a time derivative plus a nonlinear operator with parameters λ

- **forward** — data-driven solution: λ known, find the field u
- no data term: the residual and the conditions drive training
- **inverse** — data-driven discovery: λ unknown, u measured sparsely
- λ becomes a trainable variable beside $\hat{w}$, in the same gradient step
- the data term is then essential: it is what identifies λ

TALK · 2 MIN A battery cell carries three surface thermocouples. Which parameters λ of the heat equation could this network recover from them, and what would make the answer non-unique?

THE TWO SETTINGS

	forward	inverse
Raissi's name	solution	discovery
given	F, λ , conditions	F , sparse u
unknown	u	u and λ
data term	absent	essential
cost	one run per λ	one run

SENSORS three thermocouples	NETWORK $u(x, t; \hat{w})$	RESIDUAL λ trainable
---------------------------------------	--------------------------------------	--

the gradient reaches λ

Raissi et al. recover both coefficients of Burgers' equation from noisy scattered data, in one run.

Composite Loss

$$\mathcal{L} = \mathcal{L}_{data} + w_{PDE}\mathcal{L}_{PDE} + w_{DBC}\mathcal{L}_{DBC} + w_{NBC}\mathcal{L}_{NBC} + w_{IC}\mathcal{L}_{IC} + w_{IV}\mathcal{L}_{IV}$$

Raissi et al. write it as two terms, $\mathcal{L}_u$ on the data and boundary points and $\mathcal{L}_f$ on the residual; Liu splits $\mathcal{L}_u$ by the kind of condition. Same loss, six names.

SIX TERMS, EACH A MEAN SQUARED RESIDUAL

- every term is non-negative, and all vanish together at the solution
- the PDE term is always present; the others follow the problem
- the data term is empty in a forward problem
- the weights w are hyperparameters, and the terms differ in units

$$\mathcal{L}_{PDE} = \frac{1}{n_{PDE}} \sum_{i=1}^{n_{PDE}} [\mathcal{F}_{\hat{w}}(\mathbf{x}_i, t_i)]^2$$

the PDE term — the residual, squared, averaged over n interior points

WHERE EACH TERM IS EVALUATED

L_data	measurements, if any
L_PDE	interior collocation points
L_DBC	values on the boundary
L_NBC	derivatives on the boundary
L_IC	the field on $t = 0$
L_IV	its rate on $t = 0$

Soft Enforcement

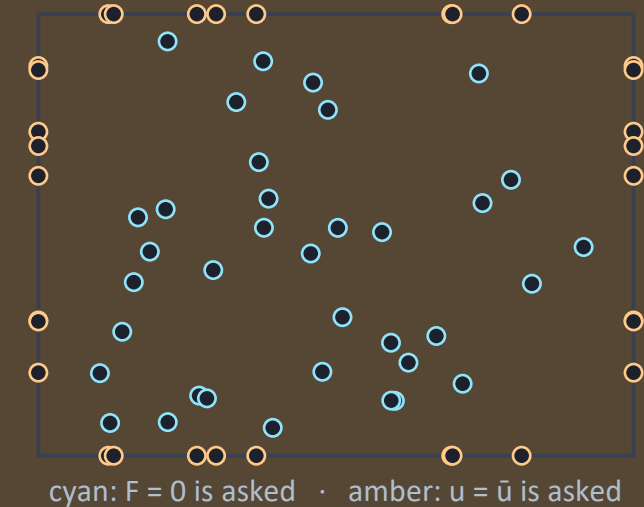
THE BOUNDARY CONDITION AS A PENALTY

$$\mathcal{L}_{DBC} = \frac{1}{n_{DBC}} \sum_{i=1}^{n_{DBC}} [u_{\hat{w}}(\mathbf{x}_i, t_i) - \bar{u}(\mathbf{x}_i, t_i)]^2$$

the prescribed value $\bar{u}$ compared with the network's output on n boundary points

- **soft** — the condition is met approximately, to the extent w buys
- it competes with the PDE term for the same weights $\hat{w}$
- when the residual wins, the network ignores its boundary
- sampled on the walls, separately from the interior set
- a wall at the coolant temperature is exactly this term

INTERIOR AND BOUNDARY SAMPLES



Two sample sets, two requests, one set of weights to satisfy both. The optimiser trades them at the rate w_{DBC} sets, and no value of w makes both exact.

Hard Enforcement

THE CONDITION BUILT INTO THE FUNCTION SPACE

$$u_{\text{trial}}(x, y) = x(1 - x) y(1 - y) \mathcal{N}_{\hat{w}}(x, y)$$

the factor vanishes on all four edges, so $u = 0$ there by construction

- the boundary term disappears — nothing left to balance
- the search space is smaller, and training is markedly easier
- the price is generality: a factor per geometry
- a curved hole needs a learned factor, a second small network

TALK · 2 MIN Removing a loss term made the error fall by two orders of magnitude. Why does one term fewer help the optimiser, and what does a plate with a hole cost you?

MEASURED · POISSON, UNIT SQUARE

	soft, w = 1	soft, w = 100	hard
relative L_2	9.9e-3	7.0e-4	3.2e-5
max error	2.2e-2	1.5e-3	5.0e-5
on the wall	2.2e-2	1.5e-3	0
PDE residual	3.8e-4	7.2e-4	6.5e-5

unit square $x(1 - x) y(1 - y)$

one wall, $x = 0$ x

a curved hole a second network, learned

in general a constrained expression (Deep-TFC)

tanh 40 x 2, 320 interior, 240 boundary, seed 0; hard is 306x more accurate than soft.

Neumann and Initial Terms

$$\mathcal{L}_{NBC} = \frac{1}{n_{NBC}} \sum_{i=1}^{n_{NBC}} [\mathcal{B}_{\hat{w}}(\mathbf{x}_i, t_i)]^2$$

$$\frac{\partial u}{\partial n} = \nabla u \cdot \mathbf{n}$$

Neumann — the residual of the normal derivative; ∇u from autograd, n the outward normal

$$\mathcal{L}_{IC} = \frac{1}{n_{IC}} \sum_{i=1}^{n_{IC}} [\mathcal{I}_{\hat{w}}(\mathbf{x}_i, 0)]^2$$

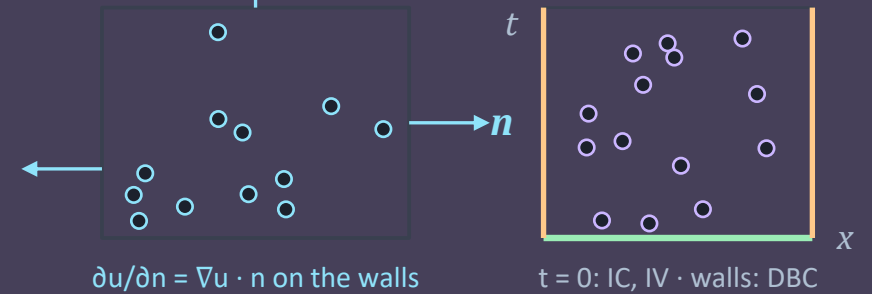
initial condition — the field at $t = 0$, against the given field

$$\mathcal{L}_{IV} = \frac{1}{n_{IV}} \sum_{i=1}^{n_{IV}} [\dot{\mathcal{I}}_{\hat{w}}(\mathbf{x}_i, 0) - \dot{u}(\mathbf{x}_i, 0)]^2$$

initial velocity — the rate at $t = 0$, from autograd, against the given rate

- **first order in time** — the initial condition only
- **second order in time** — the initial velocity too

OUTWARD NORMAL AND TIME AXIS



heat	one initial condition
wave	two: the field and its rate
a flux wall	Neumann, not Dirichlet

Initial conditions live on a slice, the boundary condition on a tube through time.

Loss Weights

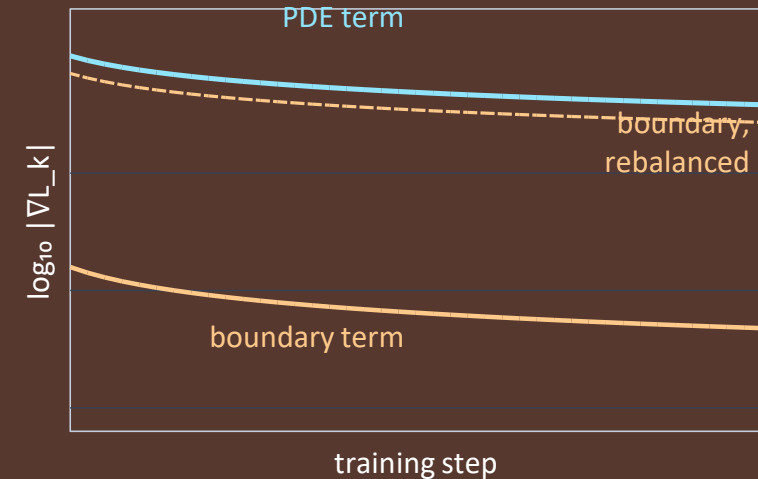
TERMS IN DIFFERENT UNITS, ONE OPTIMISER

$$w_k \leftarrow \frac{|\nabla_{\hat{w}} \mathcal{L}_{PDE}|}{|\nabla_{\hat{w}} \mathcal{L}_k|}$$

gradient-norm balancing — lift each term until its gradient matches the PDE term's

- **raw magnitudes are not comparable** — the units differ
- when one term dominates, the optimiser ignores the others
- **fixed weights** — cheap, problem-specific, fragile
- **balanced weights** — set from gradient norms as you train
- **restructuring** — enforce hard, and the awkward term is gone

GRADIENT NORM PER TERM, SCHEMATIC



After Wang et al. (2021): the PDE gradients are one to two orders larger, so the optimiser ignores the conditions. Balancing lifts amber to the dashed line.

Collocation

THREE WAYS TO EVALUATE THE RESIDUAL

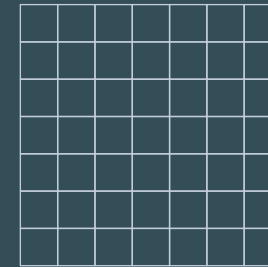
	derivatives	what it costs
strong form, autograd	exact, any order	a k-times smooth activation
strong form, stencil	finite differences	a step h, truncation error
weak form, energy	first order only	quadrature on a mesh

$$u_{xx} \approx \frac{u(x-h, y) - 2u(x, y) + u(x+h, y)}{h^2}$$

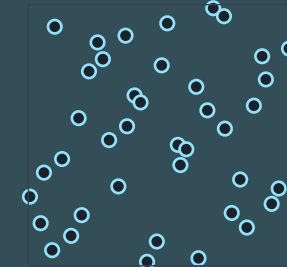
the stencil — five network evaluations per point, each differentiable in $\hat{w}$

- **collocation** — the residual is enforced at a finite set of points
- between them nothing is imposed: density limits accuracy
- **uniform, Latin hypercube or adaptive** — placement matters

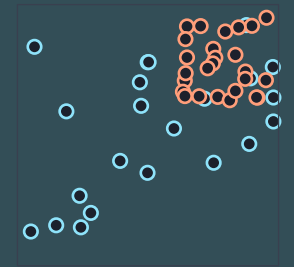
THREE WAYS TO PLACE THE POINTS



uniform grid



Latin hypercube



residual-adaptive

grid

regular; the network sees its pattern

Latin hypercube

one point per row and column

adaptive

points added where $|F|$ is largest

Ex_07 draws 320 interior points at random and 240 on the walls; orange is what refinement adds.

Energy Method

THE WEAK FORM: MINIMISE AN ENERGY, NOT A RESIDUAL

$$U(u) = \frac{1}{2} \int_{\Omega} \nabla u \cdot \nabla u \, d\Omega - \int_{\Omega} u f \, d\Omega - \int_{\partial\Omega_{NBC}} u \bar{t} \, ds$$

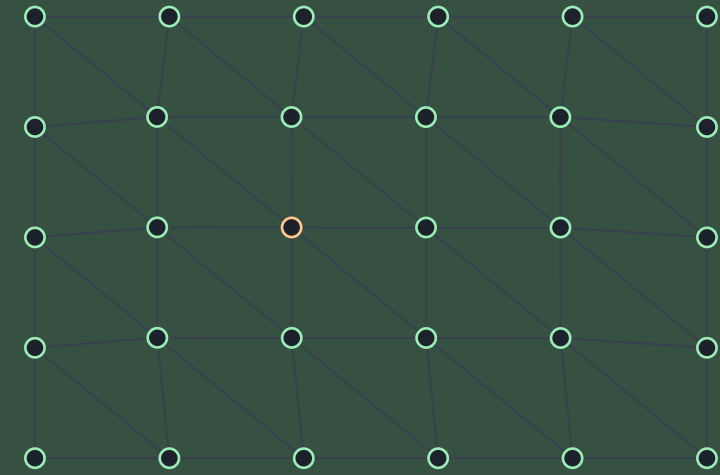
total potential energy for Poisson's equation, with a Neumann traction $\bar{t}$

$$\delta U(u) = 0 \quad \int_{\Omega} g \, d\Omega \approx \sum_{i=1}^n w_i g(\mathbf{x}_i), \quad w_i = \sum_T \frac{A_T}{3}$$

stationarity is the equation; the integral is a weighted sum over mesh nodes, each triangle giving a third of its area to each of its vertices

- integration by parts moves one derivative onto the test function
- **only first derivatives of u remain** — less asked of the activation
- Dirichlet becomes essential: it must hold for the principle to apply
- **Neumann is the boundary work term** — it needs no loss term
- the same idea with test functions is the variational PINN (VPINN)

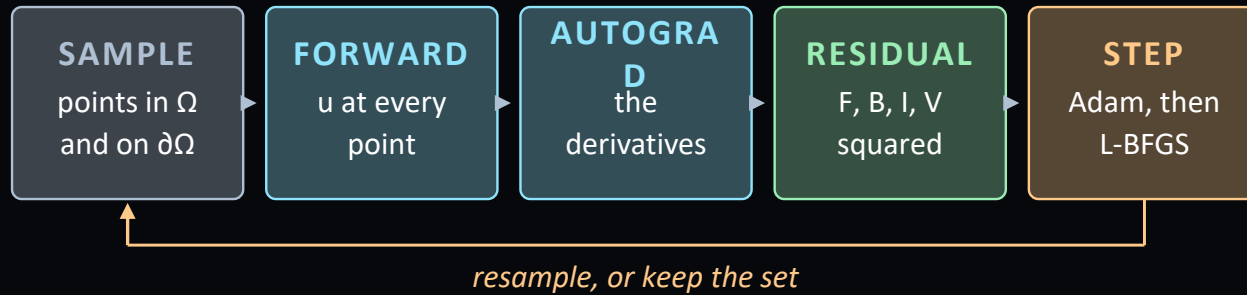
T3 TRIANGULATION AND NODAL WEIGHTS



amber node: a third of six triangles

The mesh stores no unknowns; the network is still u . The triangles only integrate, and the amber node collects a third of each triangle around it.

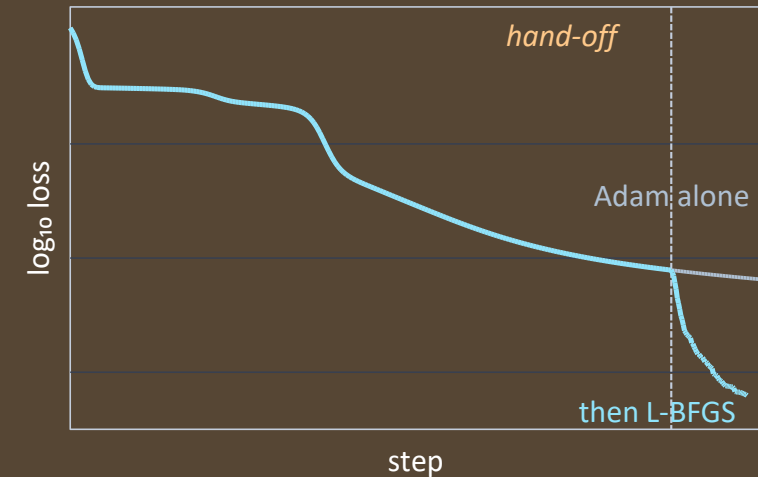
Training Loop



- **full batch** — the residual is deterministic, and cheap
- **Adam first** — robust to a poor start; stalls at a moderate residual
- **L-BFGS second** — curvature from history, two more decades

TALK · 2 MIN The cyan line drops two decades in a few hundred iterations. What in this loss makes a second-order method affordable here, and not on an image dataset?

MEASURED · ADAM THEN L-BFGS



Adam alone loss 4.2e-3, error 4.2e-4

then L-BFGS loss 3.9e-5, error 3.2e-5

Hard model, tanh 40 x 2, seed 0; switch at the plateau, not at a loss value.

Adaptive Sampling

PUT THE POINTS WHERE THE RESIDUAL IS

$$\mathbf{x}_{\text{new}} = \operatorname{argmax}_{\mathbf{x} \in S} |\mathcal{F}_{\hat{w}}(\mathbf{x})|$$

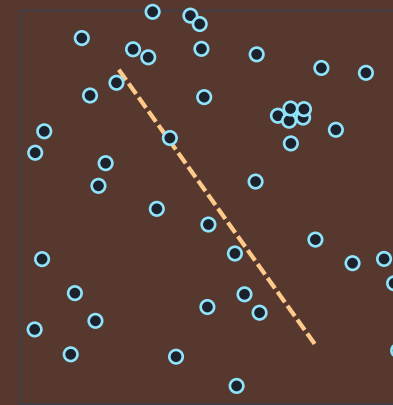
residual-adaptive refinement — add the candidate point where $|F|$ is largest, and retrain

$$\tilde{\mathcal{L}}_k = \frac{\mathcal{L}_k}{\mathcal{L}_{k^{(0)}} + \epsilon}$$

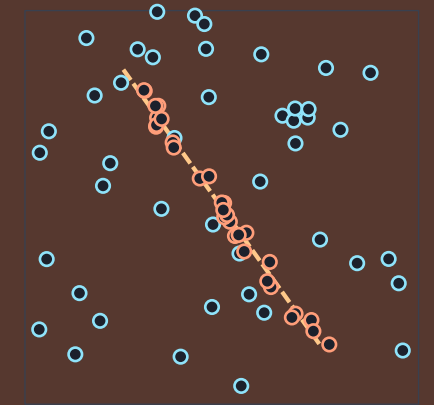
normalisation — divide each term by its initial value, so no term wins on units alone

- refinement reaches an accuracy with far fewer points than uniform
- necessary, not merely helpful, where the solution has a sharp front
- normalisation is a stabiliser; it does not fix an ill-posed problem
- **beyond both** — one network per space–time subdomain (XPINN)

REFINEMENT, BEFORE AND AFTER



44 uniform points



30 added along the front

The dashed line is a sharp front, a shock or a thermal layer. A uniform set puts three points on it; refinement puts thirty, and the residual there is what was wrong.

Challenges

GRADIENT IMBALANCE

the PDE term's gradients overwhelm the conditions · the network solves the equation and ignores the boundary · rebalance the weights, or enforce hard

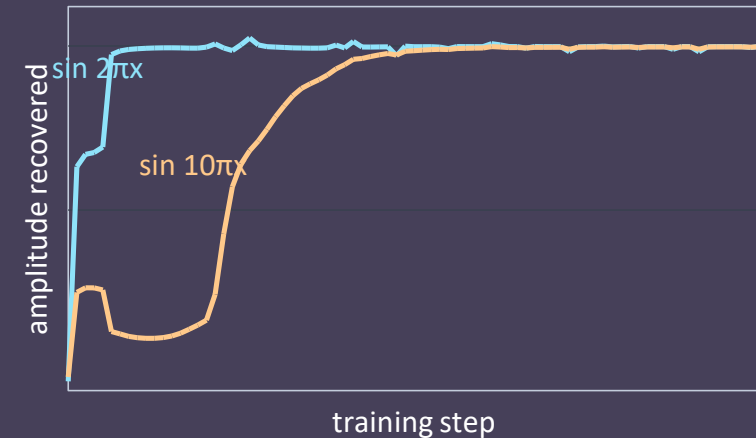
CAUSALITY VIOLATION

a time-dependent PINN fits late times before early ones · the wrong initial state is propagated forward · weight the residual so early times converge first

ADVECTION AND SHARP FRONTS

advection-dominated, discontinuous and chaotic problems are where PINNs fail · a small residual can hide a large error, or a local optimum an unstable solution · scale the variables, sin activations, adaptive sampling, or a classical solver

MEASURED · SPECTRAL BIAS



$$u(x) = \sin(2\pi x) + \frac{1}{2} \sin(10\pi x)$$

A tanh 64 × 3 network fitted to this target: the slow component arrives in a few hundred steps, the fast one two thousand later.

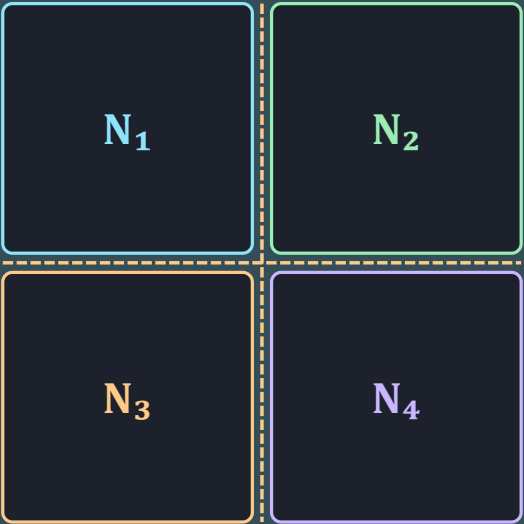
Variants

ONE NAMED METHOD PER DIFFICULTY

the difficulty	the variant	what it changes
one network is too small	XPINN, DPINN	a network per space–time subdomain
a conservation law	cPINN	a spatial split, flux matched across it
soft conditions compete	Deep-TFC, X-TFC	a constrained expression, exact by design
many geometries at once	PIPNet	PointNet features of the domain as an input
noisy data, no error bar	B-PINN	a posterior over the weights, not one fit
transport dominates	SL-PINN	learn the characteristic map, not the field

- in every row the loop of the first slide is unchanged
- what changes is the ansatz, the domain, or what is trainable
- start plain, and reach for a variant when a measurement asks for it

XPINN · ONE NETWORK PER SUBDOMAIN



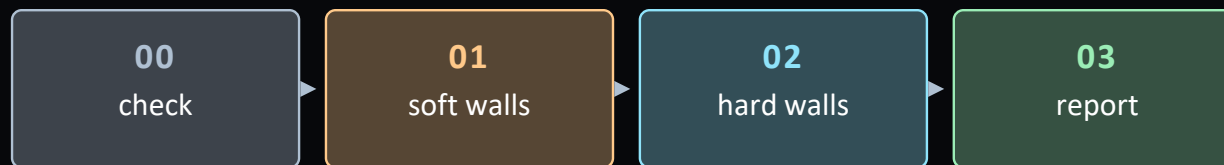
dashed: the interfaces, where the value and the flux are matched

*Four networks, four residuals, two extra terms per interface.
It parallelises.*

Exercise 7.1

FOUR NOTEBOOKS; A STATOR SLOT WITH A KNOWN ANSWER

- **00** — environment check: autograd against a hand derivative
- **01** — the walls as a penalty, and the weight sweep
- **02** — the walls built into the function space
- **03** — both under a sparse sample, and the report



Every later exercise reuses this set's three modules: `course_core`, `pinn_core` and a problem file.

THE PROBLEM

the slot	10 × 20 mm, walls at the iron temperature
the equation	$k \nabla^2 \theta + q = 0$, $\theta = 0$ on the walls
the answer	manufactured, skewed, heating everywhere
hand in	both errors, in a norm and in kelvin

REPORT QUESTION

The soft-wall error plateaus; the hard-wall error does not. Which would you use with no exact solution, and what would you check instead?

Key Takeaways

- 1 **PINN — the loss is the equation**
- 2 **Approximation — existence, not training**
- 3 **Differentiation — exact, twice**
- 4 **Formulation — one operator form**
- 5 **Loss construction — soft or hard**
- 6 **Residual evaluation — three ways**
- 7 **Training — Adam, then L-BFGS**
- 8 **Challenges — what to expect**

A dense network is the solution field; autograd gives its derivatives; the equation's residual, squared and averaged, is the loss. No labels, no mesh, and where the method sits on the bar is what the loss contains.

One hidden layer wide enough reaches any ϵ . The activation must be smooth to the order of the equation, and the sample needs at least P^* points, which counts neurons and not weights.

Autograd runs the chain rule back through the graph to machine precision; `create_graph` keeps the first pass differentiable for the second. No step size, no truncation.

$F(u; x, t, c) = 0$ on Ω with Dirichlet, Neumann, initial and velocity conditions. Forward: c known. Inverse: c trainable, and the data term is what identifies it.

A boundary term is a request; a trial solution is a guarantee. Measured: hard enforcement is two orders more accurate and removes a term to balance. Weights are rates of exchange between units.

Collocation by autograd, a finite-difference stencil on the network, or the energy of the weak form with quadrature on a mesh. Point placement sets the accuracy between points.

Full batch, deterministic, few parameters, so a second-order method is affordable. Switch at the plateau. Refine where the residual is largest; normalise each term by its start.

Gradient imbalance, causality, and advection or a sharp front, where low frequencies arrive first and a small residual can hide a large error. Six named variants answer them: XPINN and cPINN cut the domain, Deep-TFC constrains the ansatz, PIPN carries the geometry, B-PINN the uncertainty.

Next — L7.2: five equations, and which of these terms each one needs.

Questions

TEN QUESTIONS FOR THE ORAL EXAMINATION

- 1 What kind of network is a PINN, what function does it compute, and how does its loss differ from a supervised one? *Raissi et al. (2019); Wikipedia, PINNs; Liu ch. 2.1*
- 2 What does the universal approximation theorem promise for the ansatz, and what does it not? *Cybenko (1989); Liu ch. 2.1*
- 3 Why must the activation be smooth, stated from the order of the equation, and what does ReLU do to a Laplacian? *Liu ch. 4.7*
- 4 What is P^* , how does it differ from the weight count, and what does it say about the sample size? *Liu ch. 2.5.2, eq. (2.19)*
- 5 How does autograd produce $\partial^2 u / \partial x^2$, why is `create_graph` needed, and what does a finite difference cost instead? *Liu ch. 2.2.2; Ex_07 notebook 00*
- 6 Which conditions make Poisson, heat and wave well posed, and what do data-driven solution and discovery mean? *Liu ch. 2.2; Raissi et al. (2019)*
- 7 Soft against hard enforcement of a Dirichlet condition: what does each cost, and what did the measurement show? *Liu ch. 2.5.3; Ex_07 notebooks 01 – 02*
- 8 What is the weight between two loss terms, physically, and what does gradient-norm balancing do? *Liu ch. 2.2.3; Wang et al. (2021)*
- 9 Collocation, stencil or energy: what does each ask of the network, and what does each cost? *Liu ch. 2.6 – 2.8*
- 10 Why Adam then L-BFGS, when do you switch, and what is spectral bias? *Liu ch. 2.7.5; Xu et al. (2019)*

NEXT

L7.2 — Fundamental PDEs

Poisson, heat, wave, Helmholtz and Burgers: the same loop, five residuals, and the conditions each one needs to be well posed.

BEFORE THE NEXT SESSION

read	Liu ch. 2.1 – 2.7
run	Ex_07 notebooks 00 to 02
bring	the wall error, soft and hard
think about	which conditions your problem carries

Part 2 has one method and five applications. This was the method; the next lecture is the five equations it is applied to.